

Relatório de Laboratório

Laboratório de Experimentação de Software

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	LABORATÓRIO 02 - Assistentes de IA vs. codificação manual: um experimento controlado
Grupo (trio)	Luisa Clara de Paula Lara Silva · Matheus Gaston Viana Silveira · Mirelly Pego Cordeiro de Alvarenga
Link do repositório / GitHub Projects	github.com/GastonZinhow/lab-experimentacao-software · github.com/users/GastonZinhow/projects/3

1. Introdução

Ferramentas de IA generativa, como GitHub Copilot, ChatGPT, Claude e Gemini, tornaram-se onipresentes no desenvolvimento de software. Apesar disso, grande parte do que se discute sobre seu impacto em produtividade e qualidade ainda vem de relatos anedóticos, com pouca evidência controlada e reproduzível.

Neste laboratório, o grupo conduz um experimento controlado para avaliar quantitativamente os efeitos do uso de um assistente de IA (GitHub Copilot) na resolução de tarefas de programação (katas em Java), comparando-o à codificação manual quanto a tempo de resolução, defeitos e qualidade estrutural do código produzido. O experimento dá continuidade ao processo de trabalho iniciado no Lab01, com todas as tarefas e trials rastreados como Issues no GitHub Projects do grupo.

1.1 Objetivo (GQM)

Goal: analisar o uso de assistentes de IA generativa na resolução de tarefas de programação, com o propósito de comparar seu efeito frente à codificação manual, com respeito a tempo de resolução, qualidade funcional (defeitos) e qualidade estrutural do código produzido, do ponto de vista do grupo pesquisador, no contexto de katas resolvidos por estudantes de graduação sob condições controladas (crossover within-subject, time-boxed).

1.2 Questões de Pesquisa

RQ1: O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

RQ2: O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?

RQ3: O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

1.3 Hipóteses

As hipóteses foram definidas no desenho do experimento (Issue #18, arquivo lab02/docs/desenho_experimento.md), antes da execução dos trials:

RQ	Hipótese nula (H0)	Hipótese alternativa (H1)
RQ1	O uso de assistente de IA não reduz o tempo até passar nos testes.	O uso de assistente de IA reduz o tempo até passar nos testes.
RQ2	O uso de assistente de IA não reduz a quantidade de defeitos no código produzido.	O uso de assistente de IA reduz a quantidade de defeitos, aumentando a taxa de sucesso dos testes.
RQ3	O uso de assistente de IA não altera a complexidade ciclomática, a duplicação ou o LOC do código produzido.	O uso de assistente de IA altera a complexidade ciclomática, a duplicação ou o LOC do código produzido.

Informalmente, o grupo esperava que o tratamento IA fosse substancialmente mais rápido (RQ1), sobretudo porque os katas escolhidos são problemas clássicos do LeetCode, provavelmente presentes nos dados de treinamento do modelo. Para a RQ2, esperava-se pouca diferença, já que os

testes de aceitação são simples e o participante pode iterar até passar. Para a RQ3, esperava-se que o código gerado por IA fosse mais compacto e idiomático, por reproduzir soluções canônicas.

2. Metodologia

Esta seção descreve o experimento com detalhe suficiente para permitir sua reprodução. Todos os artefatos citados (katas, testes, scripts e dados) estão versionados no repositório do grupo, na pasta lab02/.

2.1 Desenho do experimento

- **Variável independente:** uso de assistente de IA, com dois níveis (tratamentos): **IA** (GitHub Copilot habilitado) e **Manual** (sem qualquer assistente de IA).
- **Variáveis dependentes:** tempo até passar nos testes (time-to-green), testes passando, testes falhando, taxa de sucesso, complexidade ciclomática, duplicação e LOC.
- **Variáveis de controle:** linguagem Java 21, JUnit 5, IDE IntelliJ IDEA Community, time-box de 35 minutos, mesmos katas e mesmo ambiente base para todos.
- **Tipo de projeto:** crossover within-subject, contrabalanceado. Cada participante resolve os 6 katas, sendo 3 com IA e 3 manualmente, com ordem de tratamentos alternada entre participantes para controlar a variação individual de habilidade.
- **Quantidade de medições:** 3 participantes $\times$ 6 katas = 18 trials (9 IA e 9 Manual).

A matriz de contrabalanceamento definida no desenho foi:

Participante	K01	K02	K03	K04	K05	K06
Matheus	IA	Manual	IA	Manual	IA	Manual
Mirelly	Manual	IA	Manual	IA	Manual	IA
Luisa	IA	Manual	Manual	IA	Manual	IA

Tabela 1 - Atribuição de tratamentos por participante e kata

Com essa matriz, todo kata é resolvido nos dois tratamentos: Matheus e Mirelly executaram tratamentos opostos em cada kata, e Luisa se soma a um dos lados. Isso permite comparar IA e Manual tanto por participante quanto dentro de cada kata.

2.2 Objetos experimentais (katas)

Foram escolhidos 6 katas (número par, permitindo a divisão 3 IA / 3 Manual), reaproveitados de problemas do LeetCode já resolvidos por um integrante no trabalho prático de FPAA (Issue #22, lab02/docs/katas.md). O grupo optou deliberadamente por uma mescla de dificuldades (fácil, média e difícil) e de técnicas algorítmicas, para observar se o efeito da IA varia com a complexidade do problema. Katas considerados inviáveis dentro do time-box de 35 minutos foram excluídos.

Kata	Problema (LeetCode)	Dificuldade	Técnica	Testes
K01	Is Subsequence	Fácil	Dois ponteiros	5
K02	Find if Path Exists in Graph	Fácil	BFS em grafo não direcionado	4
K03	House Robber	Média	Programação dinâmica	5
K04	Flower Planting With No Adjacent	Média	Guloso sobre grafo	4
K05	Longest Cycle in a Graph	Difícil	Detecção de ciclo em grafo direcionado	5
K06	N-Queens II	Difícil	Backtracking	5

Tabela 2 - Katas utilizados no experimento

Cada kata possui um esqueleto (Solution.java, lançando UnsupportedOperationException) e uma classe de testes de aceitação SolutionTest em JUnit 5, baseada nos exemplos oficiais do LeetCode, no projeto Maven lab02/code/katas-java (pacotes katas.k01 a katas.k06). Antes de qualquer implementação, todos os testes falham (0% de sucesso), confirmando que cada kata começa "vazio". No K04, como há mais de uma resposta válida, os testes verificam a propriedade da solução (nenhum par de jardins adjacentes com a mesma flor, valores entre 1 e 4) em vez de uma saída fixa.

2.3 Ambiente e ferramentas

O ambiente de execução foi fixado na Issue #23 (lab02/docs/protocolo_experimento.md), e as ferramentas de análise nas Issues #21 e #45:

Item	Configuração
Linguagem	Java 21 (maven.compiler.release=21)
Build / testes	Maven 3.6+, JUnit 5.10.2 (junit-jupiter), Maven Surefire Plugin 3.2.5
IDE	IntelliJ IDEA Community Edition, mesma versão para os três integrantes
Assistente de IA	GitHub Copilot (GitHub Student Developer Pack), integrado ao IntelliJ: autocomplete inline + Copilot Chat
Modelo do Copilot Chat	GPT-5, fixado no seletor de modelo em todos os trials IA (o autocomplete inline usa o modelo padrão, não configurável)
Métricas estáticas	CK (WMC/complexidade ciclomática e LOC; versão 0.7.0 na execução mais recente) e PMD CPD 7.27.0 (duplicação, mínimo de 50 tokens)
Cronometragem	Script próprio lab02/code/scripts/register_trial.py (Python 3.8+, sem dependências externas)
Visualização	Script lab02/code/scripts/dashboard.py (Python 3, Pandas e Matplotlib)

Tabela 3 - Ambiente e ferramentas do experimento

2.4 Protocolo de execução

Tratamento Manual: o Copilot (autocomplete e chat) é desabilitado na IDE antes de iniciar o cronômetro; é proibido consultar chatbots de IA ou pesquisar a solução do kata em buscadores, fóruns ou repositórios. São permitidos recursos não generativos da IDE (autocomplete estático, refatoração, navegação) e a documentação oficial do JDK.

Tratamento IA: o Copilot fica habilitado desde o início do cronômetro e o participante pode aceitar, editar ou rejeitar sugestões livremente. O número aproximado de prompts é registrado no campo de observação do trial.

Time-box: 35 minutos (2100 s) por trial, sem possibilidade de aumento (o script recusa valores acima de 2100 s). Ao final do tempo, o trial é encerrado independentemente do resultado; se os testes não passarem 100%, o trial é registrado como censurado em 2100 s, e não descartado.

Cada trial segue o fluxo abaixo:

- Abrir a Issue correspondente ao trial no GitHub Projects (uma Issue por participante/kata/tratamento).

- Iniciar o cronômetro: `python register_trial.py start --integrante <nome> --kata k0N --tratamento IA|Manual`.
- Resolver o kata no projeto `katas-java` sob o tratamento definido.
- Encerrar: `python register_trial.py stop --observacao "..."`. O script executa `mvn test -Dtest=katas.k0N.*Test`, lê o relatório do Surefire e grava uma linha em `lab02/code/data/raw/trials.csv` com tempo, censura, testes passando/total e taxa de sucesso.
- Salvar a solução final em `lab02/code/trials/<participante>/<tratamento>/<kata>/Solution.java` e fazer commit referenciando o número da Issue.
- Executar as métricas estáticas (a partir de `lab02/code`): `scripts/run_static_metrics.ps1 -SourcePath <solução> -RunId <participante>-<kata>-<tratamento> -CkJarPath <ck.jar> -PmdBinPath <pmd.bat>`. As saídas vão para `data/processed/ck/<run_id>/` e `data/processed/cpd/cpd-<run_id>.xml`.
- Gerar os gráficos: `python scripts/dashboard.py`, que lê `trials.csv` e as saídas do CK/CPD e grava as imagens em `data/processed/charts`.

2.5 Métricas por RQ

RQ	Métrica	Definição operacional	Ferramenta
RQ1	Tempo até passar nos testes (time-to-green)	fim – início do trial, em segundos; censurado em 2100 s se não passar; agregação por mediana e IQR	register_trial.py
RQ2	Taxa de sucesso (primária) e nº de testes falhando (complementar)	testes_passando / testes_total ao final do trial	register_trial.py + Maven Surefire
RQ3	Complexidade ciclomática	WMC da classe Solution (soma da complexidade de McCabe dos métodos) e complexidade média por método ($WMC \div n^{\circ}$ de métodos)	CK
RQ3	Duplicação	% de tokens duplicados reportados pelo CPD na solução final	PMD CPD 7.27.0
RQ3	LOC (controle)	linhas de código da classe Solution; também usada para normalizar a complexidade ($WMC \div LOC$)	CK

Tabela 4 - Métricas utilizadas para responder às RQs

A análise descritiva usa mediana e intervalo interquartil (IQR) em vez de média e desvio-padrão, dado o N pequeno e a sensibilidade da média a outliers. Para a análise inferencial, o desenho prevê o teste de Wilcoxon para amostras pareadas, coerente com o desenho within-subject.

2.6 Ameaças à validade

- **Memorização (ameaça aceita e documentada):** os 6 katas são problemas clássicos e amplamente indexados do LeetCode, com soluções públicas (inclusive no GitHub de um integrante). O Copilot pode reproduzir uma solução memorizada em vez de efetivamente "ajudar", inflando o desempenho do tratamento IA nas RQ1 e RQ2. O grupo aceitou essa ameaça conscientemente, divergindo da recomendação do enunciado de usar katas autorais ou pouco indexados.
- **Familiaridade prévia:** um integrante já havia resolvido os mesmos problemas no trabalho de FPAA, e no trial K06 IA de Luisa foi registrado que ela já conhecia o problema.

- **Medição de tempo não uniforme:** os 12 trials de Matheus e Mirelly foram cronometrados pelo `register_trial.py`, mas os 6 trials de Luisa têm tempos aproximados e autorreportados (valores redondos de 10 a 35 minutos). Além disso, o trial K04 IA de Luisa foi registrado com exatamente 2100 s (o time-box) e sem censura, e os trials K04 IA e K05 Manual têm a observação "precisou pesquisar", o que no tratamento Manual contraria o protocolo.
- **Efeito de aprendizado entre katas** e diferença real de dificuldade entre katas (a mescla fácil/média/difícil é deliberada, o que torna a comparação entre katas diferentes menos homogênea).
- **Familiaridade desigual** com Java, JUnit ou com o Copilot entre os integrantes.
- **Interrupções ou diferenças de ambiente** durante a execução, já que os trials foram executados individualmente, em máquinas próprias.

3. Resultados

3.1 Dados coletados

Foram registrados em lab02/code/data/raw/trials.csv os **18 trials planejados**: os 6 de Matheus (Issues #24 a #29), os 6 de Mirelly (Issues #30 a #35) e os 6 de Luisa (Issues #36 a #41), todos executados em 16/09/2026. Nenhum trial foi registrado como censurado e todos terminaram com 100% dos testes passando. As métricas estáticas (CK e PMD CPD) foram coletadas para as 18 soluções finais.

Participante	Kata	Tratamento	Tempo (s)	Tempo (min)	Testes	Observação
Matheus	K01	IA	83	1,38	5/5	1 prompt
Matheus	K02	Manual	1.593	26,55	4/4	-
Matheus	K03	IA	92	1,53	5/5	1 prompt
Matheus	K04	Manual	512	8,53	4/4	-
Matheus	K05	IA	61	1,02	5/5	1 prompt
Matheus	K06	Manual	1.910	31,83	5/5	-
Mirelly	K01	Manual	900	15,00	5/5	trial concluído
Mirelly	K02	IA	48	0,80	4/4	1 prompt
Mirelly	K03	Manual	982	16,37	5/5	trial concluído
Mirelly	K04	IA	69	1,15	4/4	1 prompt
Mirelly	K05	Manual	1.500	25,00	5/5	trial concluído
Mirelly	K06	IA	27	0,45	5/5	1 prompt
Luisa	K01	IA	900	15,00	5/5	tempo aproximado autorreportado
Luisa	K02	Manual	1.200	20,00	4/4	tempo aproximado autorreportado
Luisa	K03	Manual	600	10,00	5/5	tempo aproximado autorreportado
Luisa	K04	IA	2.100	35,00	4/4	tempo aproximado autorreportado; precisou pesquisar
Luisa	K05	Manual	1.800	30,00	5/5	tempo aproximado autorreportado; precisou pesquisar
Luisa	K06	IA	1.020	17,00	5/5	tempo aproximado autorreportado; já conhecia o problema

Tabela 5 - Trials registrados (trials.csv)

3.2 RQ1 - Tempo

A tabela abaixo resume o tempo até passar em todos os testes (time-to-green) por tratamento:

Tratamento	n	Mediana	Q1	Q3	IQR	Mín.	Máx.
Com IA	9	83 s (1,38 min)	61 s	900 s	839 s	27 s	2.100 s
Manual	9	1.200 s (20,00 min)	900 s	1.593 s	693 s	512 s	1.910 s

Tabela 6 - RQ1: tempo até passar nos testes por tratamento (18 trials)

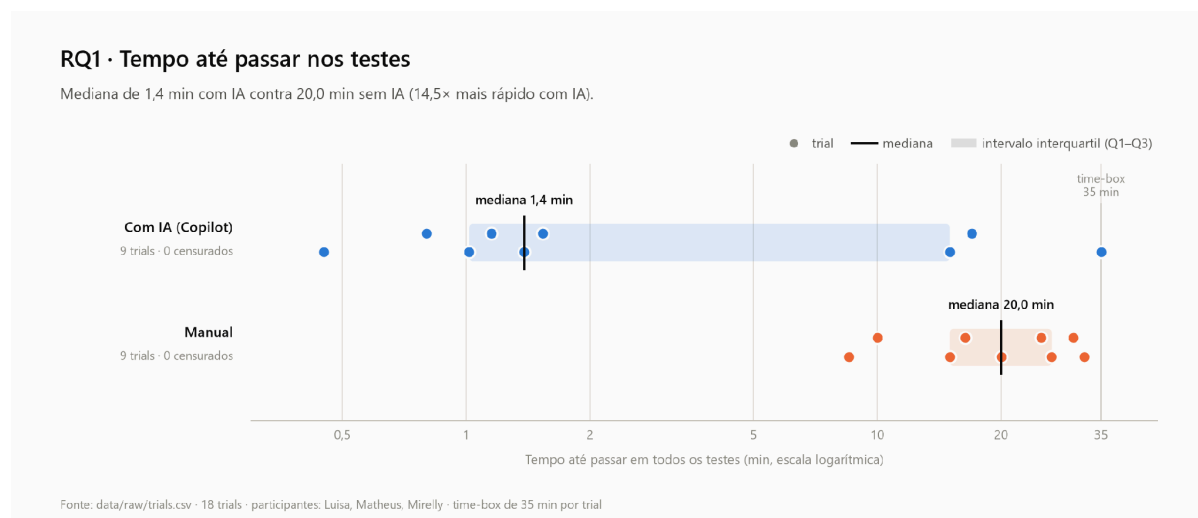


Figura 1 - RQ1: distribuição do tempo por tratamento (escala logarítmica)

A mediana do tratamento IA (1,38 min) é 14,5 vezes menor que a do tratamento Manual (20,00 min). No desenho within-subject, a comparação mais importante é dentro de cada participante: os três foram mais rápidos com IA do que sem IA, mas com magnitudes muito diferentes.

Participante	Mediana IA	Mediana Manual	Manual ÷ IA
Matheus	83 s	1.593 s	19,19×
Mirelly	48 s	982 s	20,46×
Luisa	1.020 s	1.200 s	1,18×

Tabela 7 - RQ1: mediana de tempo por participante e tratamento

Como todo kata foi resolvido nos dois tratamentos, também é possível comparar IA e Manual dentro de cada kata (mediana dos trials de cada tratamento):

Kata	Dificuldade	IA (mediana; trials)	Manual (mediana; trials)	Manual ÷ IA
K01	Fácil	491,50 s (Matheus 83; Luisa 900)	900 s (Mirelly 900)	1,83×
K02	Fácil	48 s (Mirelly 48)	1.396,50 s (Matheus 1.593; Luisa 1.200)	29,09×
K03	Média	92 s (Matheus 92)	791 s (Mirelly 982; Luisa 600)	8,60×
K04	Média	1.084,50 s (Mirelly 69; Luisa 2.100)	512 s (Matheus 512)	0,47×
K05	Difícil	61 s (Matheus 61)	1.650 s (Mirelly 1.500; Luisa 1.800)	27,05×
K06	Difícil	523,50 s (Mirelly 27; Luisa 1.020)	1.910 s (Matheus 1.910)	3,65×

Tabela 8 - RQ1: comparação IA vs. Manual em cada kata

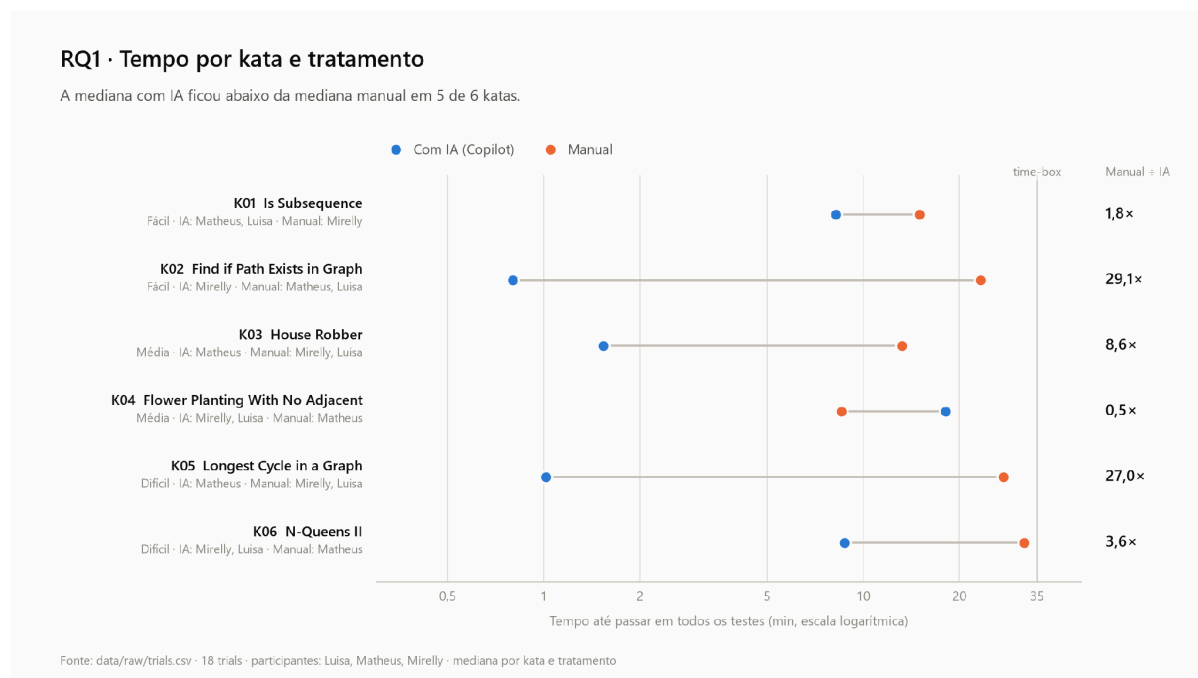


Figura 2 - RQ1: mediana de tempo por kata e tratamento

A mediana com IA foi menor em 5 dos 6 katas. A exceção é o K04, em que o trial IA de Luisa (2100 s, autorreportado) eleva a mediana do tratamento IA acima do único trial Manual (Matheus, 512 s).

Análise de sensibilidade: como os tempos de Luisa são aproximados e autorreportados, a análise foi repetida só com os 12 trials cronometrados pelo script (Matheus e Mirelly). Nesse recorte, a mediana é 1,08 min com IA e 20,68 min sem IA (19,1× mais rápido com IA), a IA é mais rápida em todos os 6 katas e os intervalos interquartis dos dois tratamentos não se sobrepõem.

Tratamento	n	Mediana	Q1	Q3	IQR	Mín.	Máx.
Com IA	6	65 s (1,08 min)	51,25 s	79,50 s	28,25 s	27 s	92 s
Manual	6	1.241 s (20,68 min)	920,50 s	1.569,75 s	649,25 s	512 s	1.910 s

Tabela 9 - RQ1: tempo por tratamento sem os trials autorreportados (12 trials)

3.3 RQ2 - Defeitos

Tratamento	Trials	Testes passando	Taxa de sucesso	Testes falhando	Trials censurados
Com IA	9	42 de 42	100%	0	0
Manual	9	42 de 42	100%	0	0

Tabela 10 - RQ2: taxa de sucesso dos testes de aceitação por tratamento

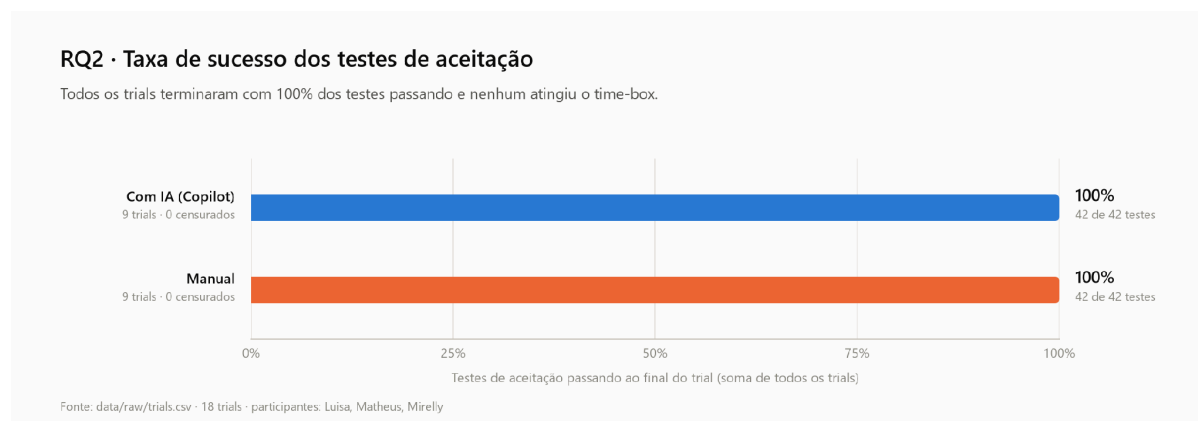


Figura 3 - RQ2: testes de aceitação passando ao final dos trials

Todos os 18 trials terminaram com 100% dos testes de aceitação passando e nenhum foi registrado como censurado. Não há, portanto, diferença observável na quantidade de defeitos entre os tratamentos: a métrica atingiu o teto (efeito teto) em ambos.

3.4 RQ3 - Estrutura do código

As métricas estáticas foram coletadas com CK e PMD CPD sobre as 18 soluções finais (saídas em lab02/code/data/processed/ck e data/processed/cpd). A tabela a seguir traz os valores por trial:

Participante	Kata	Tratament o	WMC	Métodos	Compl./méto do	LOC	WMC ÷ LOC	Duplicação
Matheus	K01	IA	5	1	5,0	13	0,38	0%
Matheus	K02	Manual	7	1	7,0	33	0,21	0%
Matheus	K03	IA	2	1	2,0	12	0,17	0%
Matheus	K04	Manual	8	1	8,0	31	0,26	0%
Matheus	K05	IA	7	1	7,0	24	0,29	0%
Matheus	K06	Manual	8	5	1,6	43	0,19	0%
Mirelly	K01	Manual	5	1	5,0	21	0,24	0%
Mirelly	K02	IA	8	3	2,7	26	0,31	0%
Mirelly	K03	Manual	4	1	4,0	18	0,22	0%
Mirelly	K04	IA	7	1	7,0	28	0,25	0%
Mirelly	K05	Manual	7	2	3,5	29	0,24	0%
Mirelly	K06	IA	7	2	3,5	26	0,27	0%
Luisa	K01	IA	5	1	5,0	12	0,42	0%
Luisa	K02	Manual	8	1	8,0	31	0,26	0%
Luisa	K03	Manual	2	1	2,0	12	0,17	0%
Luisa	K04	IA	8	1	8,0	26	0,31	0%
Luisa	K05	Manual	9	2	4,5	30	0,30	0%
Luisa	K06	IA	7	2	3,5	22	0,32	0%

Tabela 11 - RQ3: métricas estáticas por trial

Métrica	Com IA: mediana [Q1–Q3]	Manual: mediana [Q1–Q3]
WMC (complexidade ciclomática da classe)	7 [5–7]	7 [5–8]
Complexidade média por método	5,0 [3,5–7,0]	4,5 [3,5–7,0]
LOC	24 [13–26]	30 [21–31]
WMC ÷ LOC	0,31 [0,27–0,32]	0,24 [0,21–0,26]
Duplicação (% de tokens, CPD)	0% [0%–0%]	0% [0%–0%]

Tabela 12 - RQ3: resumo por tratamento (9 soluções em cada)

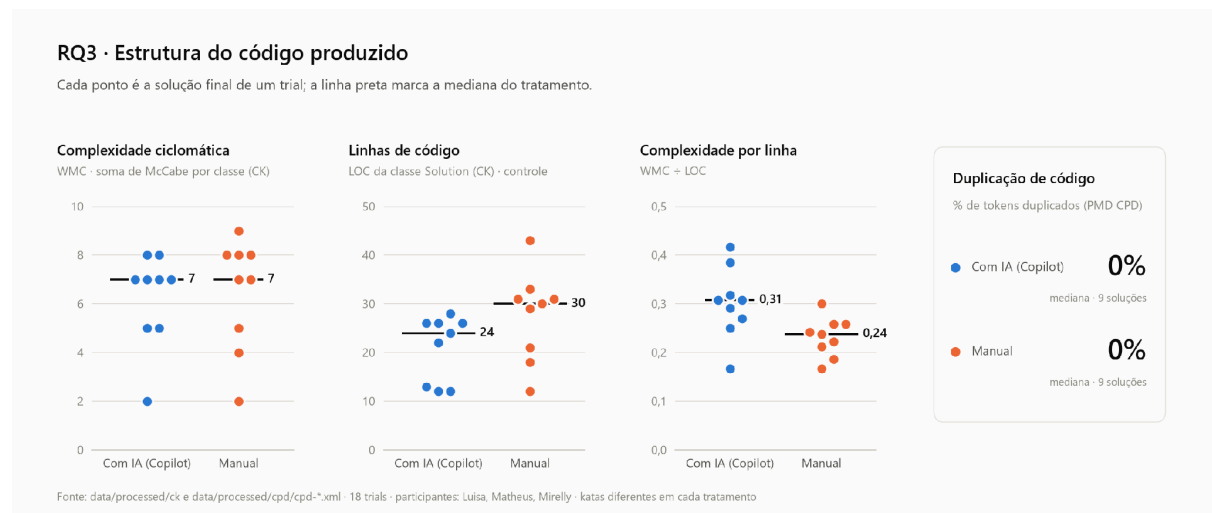


Figura 4 - RQ3: métricas estáticas por tratamento

A complexidade ciclomática ficou igual nos dois tratamentos (WMC mediano de 7 em ambos), e a complexidade média por método foi parecida. As soluções com IA foram mais curtas (mediana de 24 contra 30 LOC), o que faz a complexidade normalizada por LOC ficar maior com IA (0,31 contra 0,24 decisões por linha). O PMD CPD não encontrou nenhum bloco duplicado em nenhuma das 18 soluções.

3.5 Análise estatística e dashboard

O dashboard de visualização (Passo 6, Issue #45) foi implementado em lab02/code/scripts/dashboard.py. O script lê os dados brutos e as saídas do CK/CPD a cada execução e gera as Figuras 1 a 4 deste relatório, além de uma visão consolidada (data/processed/charts/dashboard.png).

O teste de Wilcoxon para amostras pareadas, previsto no desenho, ainda não foi executado no repositório e, por isso, não é reportado aqui. Os resultados das seções 3.2 a 3.4 são descritivos.

4. Discussão

RQ1 (Tempo): os dados descritivos são consistentes com H1. Os três participantes foram mais rápidos com IA, e a mediana geral caiu de 20,00 min para 1,38 min. A magnitude do efeito, porém, depende muito de quem executou o trial. Para Matheus e Mirelly, os trials IA duraram entre 27 e 92 segundos, sempre com 1 prompt, inclusive nos katas difíceis (K05 e K06). Esse padrão é compatível com o assistente reproduzindo soluções canônicas memorizadas de problemas clássicos do LeetCode, e não necessariamente "ajudando" o participante a raciocinar. Já os trials IA de Luisa levaram de 15 a 35 minutos, com uma diferença pequena em relação aos seus trials Manual (1,18×). Como esses tempos são aproximados e o trial K04 IA registra pesquisa externa, não é possível separar se a diferença vem de um uso distinto do Copilot ou da forma de medição. O resultado deve ser interpretado como o ganho de tempo da IA em problemas amplamente indexados, e não como estimativa geral para tarefas inéditas.

RQ2 (Defeitos): não foi possível rejeitar H0. Todos os trials terminaram com 100% de sucesso, em ambos os tratamentos. Com o time-box de 35 minutos, os participantes conseguiram iterar até passar nos testes mesmo sem IA, e os testes de aceitação, baseados apenas nos exemplos oficiais do LeetCode (4 a 5 casos por kata), não foram suficientes para diferenciar a qualidade funcional das soluções. Em uma replicação, testes com casos de borda adicionais ou um time-box menor tornariam a métrica mais sensível.

RQ3 (Estrutura): com as 18 soluções, não há indício de que a IA altere a complexidade ciclomática: o WMC mediano é o mesmo nos dois tratamentos e não houve duplicação em nenhuma solução. A diferença aparece no tamanho: o código com IA é mais curto e, portanto, mais denso em decisões por linha. Ou seja, a IA tende a produzir soluções mais enxutas, e não estruturalmente mais simples. Como cada kata tem uma estrutura natural diferente e o número de soluções por tratamento é pequeno (9), essa diferença de LOC deve ser confirmada pela análise inferencial.

Limitações gerais: o experimento conta com 18 trials de três participantes e katas de dificuldade propositalmente heterogênea. Além da memorização, pesam sobre os resultados a familiaridade prévia com os problemas (trabalho de FPAA e o K06 de Luisa), a medição não uniforme do tempo (6 trials autorreportados, um deles no limite do time-box sem censura) e os registros de pesquisa externa em dois trials de Luisa. Os resultados devem ser lidos como evidência descritiva e exploratória, a ser confirmada pelo teste de Wilcoxon.

Considerações finais: mesmo com as ressalvas, o experimento mostra de forma consistente que, para problemas algorítmicos clássicos e bem conhecidos, um assistente de IA reduz fortemente o tempo de resolução sem prejuízo da correção funcional medida pelos testes, e sem alterar a complexidade do código produzido. A principal lição metodológica do grupo é que a escolha dos objetos experimentais domina o resultado: com katas amplamente indexados, o experimento tende a medir a capacidade de recuperação do modelo mais do que o apoio à resolução de problemas. A segunda lição é a importância de medir todos os trials com o mesmo instrumento: os trials autorreportados são justamente os que mais divergem do restante da amostra. Uma replicação com katas autorais e cronometragem uniforme seria o próximo passo mais valioso.

5. Repositório e GitHub Projects

Repositório: <https://github.com/GastonZinhow/lab-experimentacao-software>

GitHub Projects: <https://github.com/users/GastonZinhow/projects/3>

Artefatos do Lab02 no repositório:

- lab02/docs/desenho_experimento.md: desenho, hipóteses e ameaças à validade (Issue #18).
- lab02/docs/katas.md: seleção e validação dos katas (Issue #22).
- lab02/docs/protocolo_experimento.md: ambiente, assistente de IA e protocolo (Issue #23).
- lab02/docs/metricas_estaticas.md e lab02/code/scripts/run_static_metrics.ps1: coleta de CK e PMD CPD (Issue #21).
- lab02/code/katas-java: projeto Maven com esqueletos e testes de aceitação (Issues #19 e #22).
- lab02/code/scripts/register_trial.py: cronometragem e registro dos trials (Issue #20).
- lab02/code/scripts/dashboard.py e lab02/code/data/processed/charts: dashboard de visualização (Issue #45).
- lab02/code/data/raw/trials.csv, data/processed/ck e data/processed/cpd: dados brutos e métricas estáticas.
- lab02/code/trials: soluções finais de cada trial (Issues #24 a #41).

6. Referências

GITHUB. GitHub Copilot Docs. Disponível em: <https://docs.github.com/en/copilot>.

ANICHE, M. CK: Java code metrics calculator. Disponível em: <https://github.com/mauricioaniche/ck>.

PMD. Copy/Paste Detector (CPD). Disponível em: <https://pmd.github.io/>.

LEETCODE. Problemas Is Subsequence, Find if Path Exists in Graph, House Robber, Flower Planting With No Adjacent, Longest Cycle in a Graph e N-Queens II. Disponível em: <https://leetcode.com/problems/>.

THE PANDAS DEVELOPMENT TEAM. pandas. Disponível em: <https://pandas.pydata.org/>.

HUNTER, J. D. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, v. 9, n. 3, p. 90-95, 2007.

WOHLIN, C. et al. Experimentation in Software Engineering. Springer, 2012.